
MEDICAL HEURISTIC LEARNING: AN LLM-DRIVEN FRAMEWORK FOR INTERPRETABLE AND AUDITABLE CLINICAL DECISION RULES

Wei Xu^{1,*}, Ke Yang^{2,*,†}, Gang Luo¹, Keli Zheng⁴, Lingyan Hu⁵, Jing Wang^{3,†}, Kefeng Li^{1,†}

¹Centre for Artificial Intelligence Driven Drug Discovery, Macao Polytechnic University, Macao SAR

²Key Laboratory of Short-Range Radio Equipment Testing and Evaluation, Ministry of Industry and Information Technology Terahertz Science Application Center (TSAC), Beijing Institute of Technology, Zhuhai, China

³Department of Critical Care Medicine, Yantai Yuhuangding Hospital, Qingdao University, Yantai, Shandong, China

⁴The University of Hong Kong, Hong Kong SAR

⁵College of Information Engineering, Dalian University, Dalian, Liaoning, China

*These authors contributed equally to this work.

[†]Corresponding authors: kefengl@mpu.edu.mo (K. Li), jinga@qdu.edu.cn (J. Wang), yangke_1208@163.com (K. Yang).

ABSTRACT

Predictive modeling for clinical tabular data is central to clinical decision support and therefore requires not only strong predictive performance but also transparent decision logic. Although deep learning and tree-based ensemble methods can achieve high accuracy, their black-box nature remains a major obstacle to clinical deployment. This challenge is further compounded by common characteristics of medical data, including limited sample sizes, severe class imbalance, and feature evolution arising from changes in diagnostic criteria and clinical documentation. To address these issues, we propose Medical Heuristic Learning (MHL), an instantiation of the learning-beyond-gradients paradigm for clinical tabular prediction. Instead of relying on neural network weight updates, MHL uses a large language model (LLM)-driven workflow that integrates statistical probes, medical knowledge probes, rule synthesis, and code-level iterative refinement to optimize a deterministic and executable decision system. The resulting model is expressed not as opaque parameters, but as versioned pure-Python decision rules that are explicitly interpretable, fully auditable, and clinically grounded. MHL also supports continual learning by starting from previously validated rules and iteratively revising them using updated feature information under data drift or feature evolution. Comprehensive experiments on medical datasets show that MHL achieves performance comparable to state-of-the-art methods while maintaining strong behavior in small-sample and highly imbalanced settings. The results further indicate that this explicit rule update mechanism can help alleviate catastrophic forgetting under feature evolution. Overall, these findings suggest that non-gradient-based heuristic systems offer a transparent and adaptable alternative for high-stakes clinical decision support.

1 Introduction

Machine learning models are increasingly being explored for clinical decision support [1]. In many applications, they promise earlier risk stratification, more efficient allocation of clinical resources, and more timely intervention. In medicine, however, predictive accuracy alone is not sufficient. Clinical decisions are made in high-stakes settings, where erroneous predictions may lead to delayed treatment, inappropriate escalation of care, or unnecessary intervention. Clinicians therefore cannot reasonably rely on a system whose decision process is opaque. This creates a persistent tension between predictive performance and interpretability in the deployment of clinical prediction models. Simpler models, such as linear models or shallow decision trees, are easier to inspect but are often perceived as less competitive on complex tasks. By contrast, deep neural networks, ensemble tree models, and other black-box approaches often deliver stronger predictive performance, yet their post-hoc explanation tools, including SHAP and LIME, remain sensitive to modeling choices and input perturbations, limiting their ability to provide stable and clinically reliable insight [2, 3, 4, 5].

Beyond limited interpretability, current state-of-the-art models remain vulnerable to the statistical and temporal complexity of clinical data. Medical datasets are often small, imbalanced, and heterogeneous rather than large, clean, and balanced. In such settings, models that rely on gradient-based optimization or greedy partitioning can become unstable. A further challenge is that clinical practice is inherently dynamic. As medical knowledge advances and diagnostic technologies evolve, both the set of measured variables and the criteria used to define disease may change over time [6, 7]. When data drift or feature evolution occurs, conventional black-box models are often difficult to adapt directly to the new feature space. Moreover, updating these models through transfer learning can induce catastrophic forgetting, whereby previously acquired knowledge is partially overwritten during adaptation to new data.

To address this impasse, we bring the emerging perspective of learning beyond gradients, namely Heuristic Learning, into clinical prediction. Traditional heuristic algorithms search for efficient and feasible solutions in a candidate solution space by iteratively refining rule-based strategies or bio-inspired mechanisms, such as particle swarm optimization and ant colony optimization, without relying on gradient information [8, 9]. Heuristic Learning extends this intuition by introducing the large language model as a rule generation agent that can draft, revise, and evolve executable heuristic rule modules, thereby shifting optimization from a numerical parameter space to an explicit program space [10]. In this view, learning is achieved not primarily by repeatedly updating hidden parameters, but by explicitly improving an executable rule system in response to feedback. Rather than encoding medical knowledge implicitly in model weights, this perspective treats rules, procedures, and program structure as the main objects of optimization. Such a formulation is particularly well suited to clinical deployment. When a predictive system is expressed directly as readable and executable code, its reasoning process can be inspected, audited, and revised in a way that is far more compatible with medical accountability than conventional black-box models. Heuristic Learning therefore provides a natural conceptual basis for developing white-box clinical decision rules that are both transparent and operational.

Building on this perspective, we propose Medical Heuristic Learning (MHL), an LLM-driven framework designed specifically for clinical tabular prediction. Unlike earlier explorations of heuristic learning systems that relied heavily on open-ended coding agents, MHL is organized as a structured and deterministic workflow tailored to the constraints of medical applications. The workflow consists of four stages. First, a statistical probe extracts objective distributional signals from the training data and identifies candidate predictive patterns. Second, a medical knowledge probe introduces domain priors so that the resulting rules are informed not only by empirical associations but also by clinically meaningful considerations. Third, an initial rule generation stage uses the large language model as a constrained rule synthesis engine to produce executable Python decision rules with explicit comments and clinical rationale. Fourth, a rule iteration stage refines these rules through closed-loop analysis of errors and degradations, enabling small, auditable code-level updates rather than wholesale rewrites. Importantly, when data drift or feature evolution occurs, MHL continues learning by starting from the previously validated rule set and incorporating changed feature information into a new round of explicit rule revision. Adaptation is thus achieved through controlled modification of white-box logic rather than through overwriting hidden parameters, providing a direct mechanism for mitigating catastrophic forgetting.

We evaluate MHL on multiple medical datasets, including UK Biobank (UKB) [11], the Critical Care Information Database (CCID), and the Medical Information Mart for Intensive Care (MIMIC) [12]. The experiments show that MHL achieves performance comparable to contemporary black-box state-of-the-art methods while preserving full white-box transparency. More importantly, it demonstrates strong robustness in small-sample and highly imbalanced settings, where clinically useful models are often hardest to obtain. In addition, under a simulated continual-adaptation setting involving data drift and feature evolution, the explicit rule revision mechanism of MHL effectively mitigates catastrophic forgetting. Taken together, these findings suggest that MHL offers a practical alternative for high-stakes clinical decision support by combining interpretability, auditability, and adaptability within a unified predictive framework.

2 Results

2.1 Introduction to Medical Heuristic Learning (MHL) and Experimental Design

We begin by summarizing the overall workflow of Medical Heuristic Learning (MHL). As shown in Figure 1, subfigure (a) presents the standard MHL pipeline. The workflow starts from raw tabular data, feature descriptions, a task description, and a metric priority description, and then derives two complementary forms of evidence in parallel. The statistical probe extracts empirical distributional signals, including univariate associations and descriptive summaries, whereas the medical knowledge probe provides clinically grounded interpretations and candidate thresholds linked to feature semantics. These two evidence streams jointly constrain the generation of the initial executable v0 Python rule and continue to guide subsequent revisions through explicit analyses of errors, degradations, and version trajectories. MHL therefore does not learn by absorbing information into hidden parameters. Instead, it improves a white-box rule system through probe-constrained rule generation and explicit code-level revision. The methodological details of these components are described in Section 4.

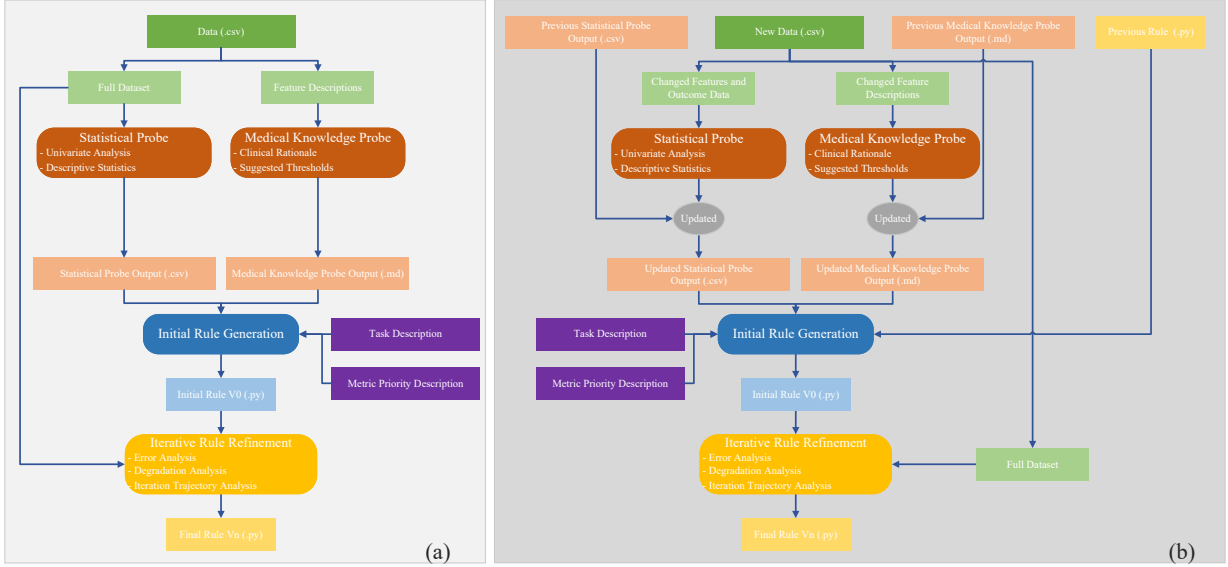


Figure 1: Overview of the Medical Heuristic Learning workflow. (a) shows the standard rule generation pipeline, and (b) shows the continual learning pipeline under feature evolution.

MHL is further designed to extend beyond one-shot static modeling and to support continual learning, as illustrated in subfigure (b) of Figure 1. When the available feature space changes, the framework preserves validated outputs from the previous stage, including probe results and rule code, and uses them as explicit priors for subsequent adaptation. It identifies newly introduced, removed, and retained features, updates the statistical and medical probes accordingly, and resumes rule generation and iteration from the existing white-box rule base while conditioning the updated stage on refreshed task and metric descriptions. Continual learning in MHL is therefore implemented as evidence-guided revision of an existing rule system rather than wholesale overwriting of hidden parameters. The corresponding methodological details are likewise provided in Section 4.

Within this framework, we evaluated MHL in three medical prediction settings. UK Biobank (UKB) used blood metabolomic measurements as predictors for depression prediction [11], whereas the Critical Care Information Database (CCID) [13] and the Medical Information Mart for Intensive Care (MIMIC) [12] used ICU admission measurements as predictors for 28-day mortality. The baseline models covered major linear, tree-based, and deep tabular paradigms, including Logistic Regression [14], Decision Tree [15], XGBoost [16], LightGBM [17], MLP [18], and FT-Transformer [19]. All experiments were repeated with three random seeds, and unless otherwise specified, we report averaged results. Except for the dedicated backend comparison in Section 2.6, all experiments used DeepSeek-V4-Pro as the underlying LLM backend. The remainder of this section first presents a rule generation case study and then reports ablation, sample-size, class imbalance, backend comparison, and continual learning experiments.

2.2 Case Study of Rule Generation and Iteration

To illustrate what MHL actually produces, we first present a representative case from the CCID dataset with 3000 training samples. Figure 2 shows a complete white-box artifact chain, including the task description, the metric priority description, excerpts from the statistical probe, excerpts from the medical knowledge probe, the initial rule v0 with its generation rationale, and the final selected rule v9 with its revision rationale. Taken together, these artifacts show how task requirements, evaluation priorities, empirical evidence, and clinical prior knowledge are organized into executable logic, and how validation feedback is subsequently converted into explicit grounds for rule revision. The output of MHL is therefore not merely a clinical decision rule, but an interpretable and auditable rule system with a traceable generation rationale, revision rationale, and versioned refinement trajectory.

Figure 3 further shows the performance trajectory across rule versions. In this case, the two probes initially yielded a rule with a strongly one-sided decision tendency. This pattern is visible in the starting metrics, where v0 showed very high sensitivity of approximately 0.95 but very low specificity of approximately 0.15, while ACC remained modest. Subsequent iterations were then driven by validation feedback. The first few revisions mainly corrected this severe specificity deficit, bringing the rule back from an almost always-positive prediction pattern, whereas later revisions

refined the balance among ACC, F1, sensitivity, and specificity through small and explicit code changes. By v9, the rule had reached a more appropriate overall trade-off, and the system selected it as the final version. This case provides an intuitive white-box reference for the quantitative experiments reported below.

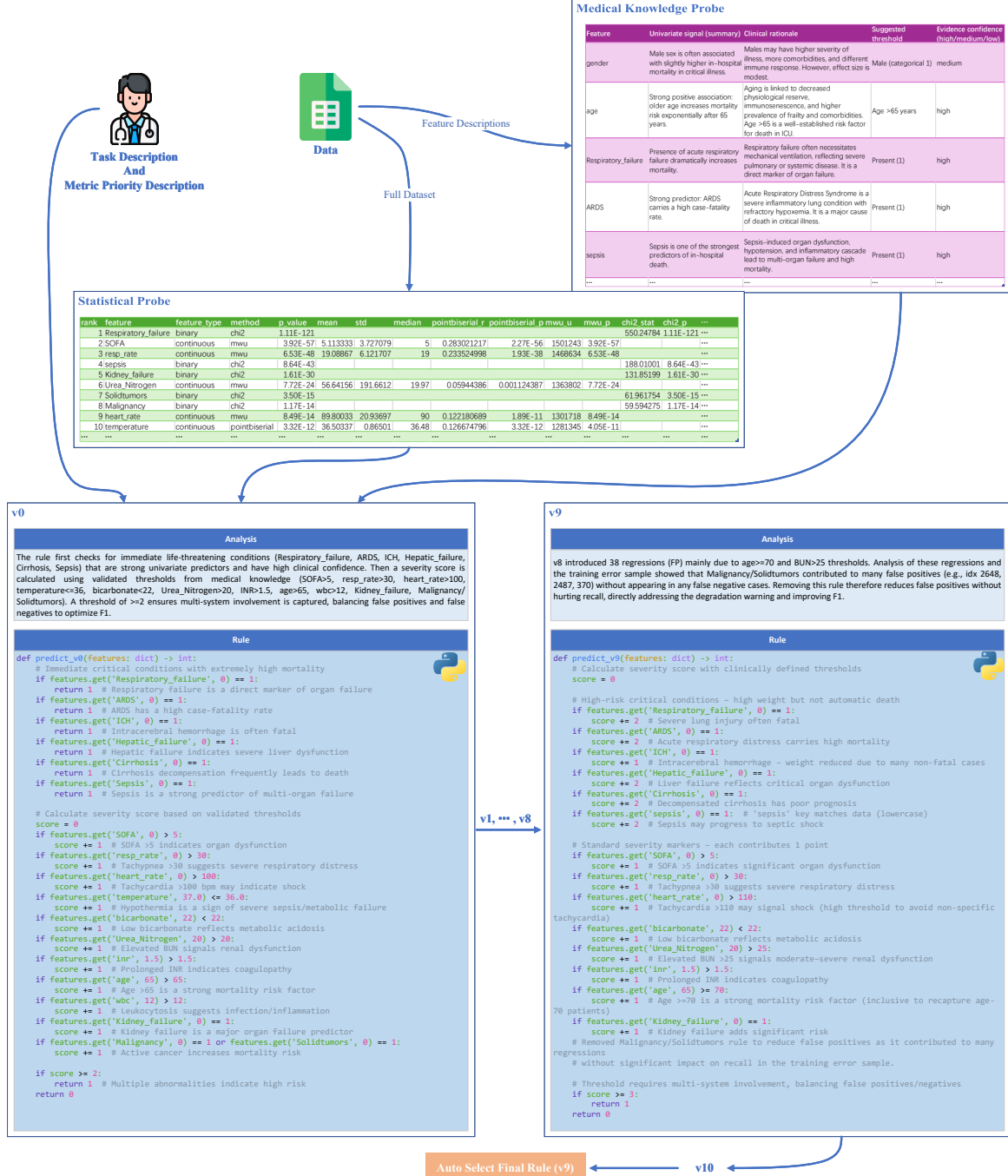


Figure 2: Case study of MHL rule generation and revision on the CCID dataset, showing probe outputs, the initial rule, and the final selected rule with revision notes.

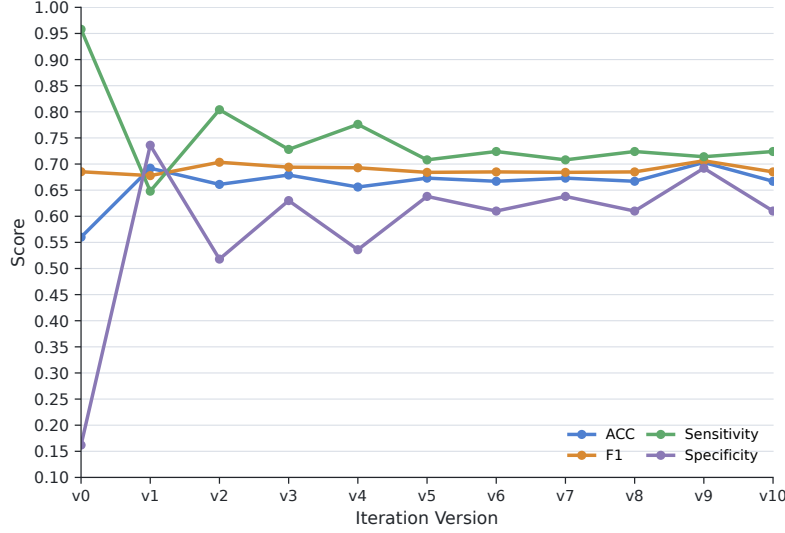


Figure 3: Evolution of ACC, F1, precision, and recall during rule iteration for the CCID case study.

2.3 Ablation Study on Probe Mechanisms

To assess how the probe modules contribute to rule quality, we conducted a systematic ablation study within the MHL workflow. The experiment was performed on UKB and CCID, with validation and test sets each containing 1000 samples under 1:1 positive-negative balanced sampling, and with training sizes ranging from 10 to 3000. Under a fixed workflow and fixed LLM backend, we compared four configurations: no probe (N), knowledge probe only (K), statistical probe only (S), and the full dual-probe configuration (S+K). Results are summarized in Figure 4 and Table 3. In the figure, thick lines denote mean trends across three random seeds, thin lines denote seed-specific trajectories, and shaded bands denote the range of variation across seeds.

Across settings, S+K did not always achieve the highest score at every operating point, but it delivered the most stable and least failure-prone performance overall. In Figure 4, N and S show greater dispersion across random seeds, with particularly wide shaded bands in the low-sample regime. This pattern is consistent with stronger sampling variability under small training sets. When the sampled subset is less representative of the underlying distribution, a rule generator that relies only on sparse empirical signals is more likely to encode local noise as if it were a stable pattern. The introduction of K reduced this variability by supplying clinical priors and threshold references, thereby limiting the extent to which rule generation followed accidental small-sample regularities. When empirical evidence and medical priors were combined, the S+K curves became smoother and the seed-to-seed range became narrower. Table 3 supports the same conclusion, showing that S+K maintained a consistently usable F1 level across the examined sample sizes.

The ablation results further suggest that different probe settings bias the model toward different sensitivity-specificity trade-offs. Overall, K alone more often shifts the rules toward higher sensitivity, usually at the cost of specificity, whereas S alone more often favors specificity in better-sampled settings and may reduce sensitivity. These patterns should be interpreted as general tendencies rather than rules that hold in every setting. By contrast, S+K delivered the most appropriate overall balance in most settings and was more consistent in both stability and aggregate performance. Taken together, the results suggest that the two probes are distinct but complementary. The statistical probe prioritizes features that stand out in the observed data, whereas the medical knowledge probe organizes thresholds and rule structure in a more clinically grounded manner. On this basis, S+K was adopted as the default configuration in the subsequent experiments.

2.4 Robustness Across Different Sample Sizes

We next examined how MHL behaves across different training sample sizes, because reliable performance under limited labeled data is particularly important in medical tabular prediction. This experiment was conducted on UKB and CCID, again with balanced validation and test sets of 1000 samples each, while the training set size increased from 10 to 3000. MHL was compared with Logistic Regression, MLP, Decision Tree, XGBoost, LightGBM, and FT-Transformer. Overall trends are shown in Figure 5, and detailed metrics are reported in Table 4.

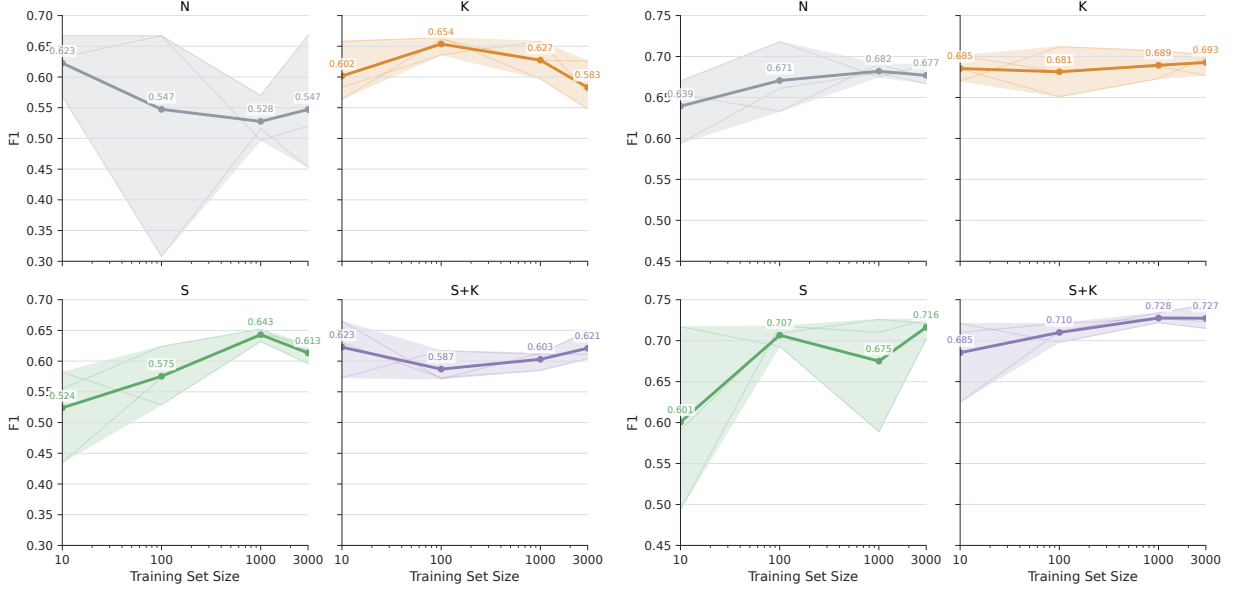


Figure 4: Ablation results on UKB (Left) and CCID (Right).

The clearest advantage of MHL appeared in the low-resource regime. On both UKB and CCID, MHL consistently achieved the best F1 whenever the training size was below 100. On UKB, Table 4 shows that MHL reached an F1 of 0.623 at $n = 10$ and outperformed the main baselines under the same setting. By contrast, LightGBM collapsed to an F1 of 0, and FT-Transformer exhibited a degenerate single-class pattern, with sensitivity and specificity fixed at 1 and 0. A similar trend was observed on CCID, where MHL remained ahead of most baselines throughout the small-sample range. These results suggest that medically informed priors and explicit rule structure can partly compensate for the fragility of purely statistical learners when labeled data are scarce.

As the training size increased, some black-box models became more competitive by exploiting richer statistical structure in the data. This pattern is visible in the larger-sample region of Figure 5 and in Table 4, where tree-based models eventually matched or surpassed MHL in some settings. MHL, however, did not become unstable as sample size changed. Instead, it maintained a smooth and usable performance trajectory across the full range. Thus, in data-rich settings, the primary value of MHL is no longer absolute dominance in predictive score, but the ability to retain competitive performance while preserving a fully transparent and auditable model form.

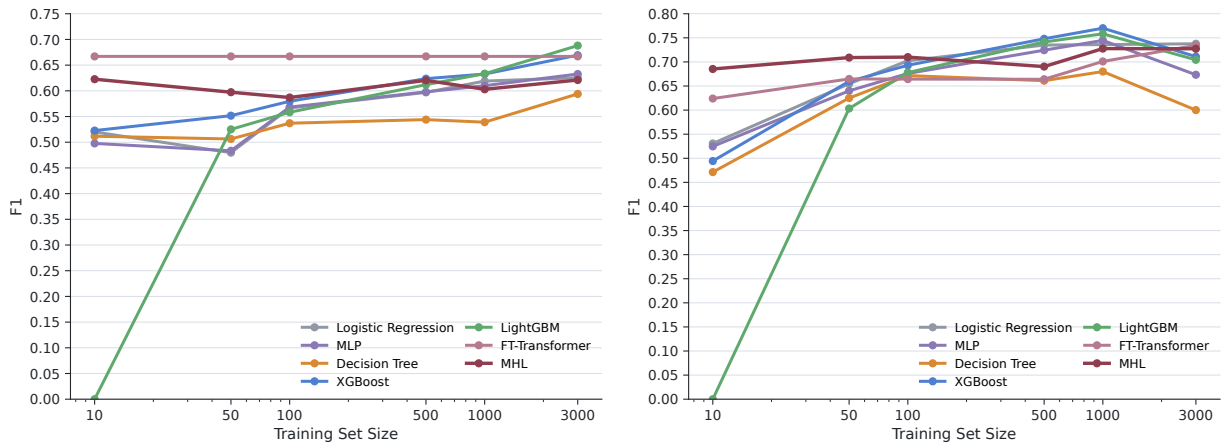


Figure 5: Performance across different training set sizes on UKB (Left) and CCID (Right).

2.5 Performance under Extremely Imbalanced Distributions

Beyond sample size, severe class imbalance represents another major challenge in clinical prediction. To evaluate this setting, we kept the validation and test sets balanced at 1:1 while systematically varying the positive-to-negative ratio in the training data from 1:1 to highly skewed settings such as 50:1 and 1:50. The experiment was conducted at training sizes of 1000 and 3000. Because the central question was whether models could still detect the minority class under highly biased training signals, we interpreted the results jointly using the confusion-matrix summaries in Figure 6 and the detailed metrics in Table 5.

On both UKB and CCID, most baselines behaved better at $n = 3000$ than at $n = 1000$, indicating that additional training data could partially mitigate the effect of imbalance. This mitigation, however, remained limited under extreme ratios. In highly skewed settings, many baselines drifted toward one-sided prediction, a pattern that was especially evident on UKB. For example, Logistic Regression, MLP, and XGBoost could still produce F1 scores around 0.667 in some skewed settings, even though Table 5 shows that their specificity was already close to 0. At the opposite end of the ratio spectrum, several baselines deteriorated to near-zero F1. Taken together, Figure 6 and Table 5 indicate that, under severe imbalance, many black-box baselines can retain superficially acceptable summary scores while already losing meaningful minority-class discrimination.

MHL showed a markedly different behavior under extreme imbalance. In several of the most severely imbalanced and difficult settings, its confusion matrices remained substantially more usable. As shown in Figure 6, MHL did not collapse at either 50:1 or 1:50 for CCID at both $n = 1000$ and $n = 3000$, nor for UKB at $n = 1000$. In these cases, the confusion matrices still retained meaningful counts on both classes rather than degenerating into near-one-sided prediction, indicating that the model preserved a workable balance between minority-class detection and majority-class control even under highly distorted training distributions. At the same time, MHL was not immune to failure under all extreme conditions. In one notable case, UKB at $n = 3000$ with a 1:50 training ratio, the MHL confusion matrix also collapsed, but in a different direction from most baselines. Rather than being dominated by the majority class, it became overly biased toward the minority class. This behavior may reflect the method’s explicit emphasis on correcting errors and preventing degradations during rule iteration. Even with this exception, the overall level of performance and stability achieved by MHL under severe imbalance remained clearly stronger than that of the baseline models.

2.6 Impact of LLM Backends

After examining the effects of data conditions, we evaluated how strongly MHL depends on the underlying LLM backend. In this experiment, the training set was fixed at $n = 1000$ with 1:1 balanced sampling, the validation and test sets were also kept balanced, and the probe configuration, rule generation procedure, and iteration process were unchanged. Only the LLM backend was replaced. The compared models included GPT-5.5, Gemini 3.1-Pro, DeepSeek V4-Pro, DeepSeek V4-Pro-Thinking, DeepSeek V4-Flash, and Qwen 3.7-Max. Results are summarized in Figure 7 and Table 6.

The results indicate that the LLM backend influenced both the performance ceiling and the decision style of the resulting rules. No single backend dominated across all datasets: GPT-5.5 performed best on UKB, whereas Gemini 3.1-Pro led on CCID, with several other strong backends remaining close. Beyond the final score, the backend also changed how the synthesized classifier traded sensitivity against specificity. For example, GPT-5.5 on UKB produced a more aggressive high-sensitivity rule, with sensitivity reaching 0.899 but specificity only 0.192. DeepSeek V4-Pro, by contrast, achieved a lower overall score but produced a more balanced sensitivity-specificity profile. These results suggest that the LLM backend shapes not only how well the rule performs, but also what type of decision boundary the rule search process tends to synthesize.

The backend comparison further suggests that MHL remains usable across different LLMs. Contrary to the common expectation that an explicit reasoning mode would necessarily improve rule generation, DeepSeek V4-Pro-Thinking did not improve the final results over its non-thinking counterpart. A possible explanation is that MHL already decomposes the task into structured probes, constrained rule generation, and versioned refinement; under this workflow, longer chain-of-thought-style reasoning does not automatically translate into better executable rules and may even introduce revisions that are unnecessarily complex or less well calibrated. More broadly, the statistical probe, medical knowledge probe, and versioned rule refinement process appear to impose sufficiently clear constraints on the rule generation task, so that rules generated by different LLMs remain usable. Thus, MHL can benefit from stronger foundation models while retaining portability across LLM backends.



Figure 6: Confusion-matrix summaries under different imbalance ratios for UKB (Left) and CCID (Right).

2.7 Continual Learning and Adapting to Feature Evolution

Finally, we extended the evaluation from static prediction to continual learning under feature evolution, testing whether MHL can preserve prior knowledge while incorporating new variables. In sepsis assessment, clinical criteria shifted from SIRS-based definitions toward SOFA-based definitions in the Sepsis-3 consensus [7]. One practical consequence of this transition is that some healthcare units may stop recording SIRS-related variables and begin recording SOFA-related variables instead, creating a realistic form of feature evolution in clinical data collection. We simulated this transition on the Medical Information Mart for Intensive Care (MIMIC) through a two-stage task. In Stage 1, models were trained on the older feature set that included SIRS, with a training size of 1000. In Stage 2, they were required to adapt to a new feature set that included SOFA, with only 40 training samples available. The validation and test set sizes in each stage were 500 and 800, respectively. Stage-wise results are shown in Figure 8 and Table 7.

The baseline models showed a broadly consistent pattern of degradation when the available feature space changed. In this setting, each baseline started from the model obtained in Stage 1 and was further trained after being transferred into the new Stage 2 feature space. Despite this continual-training setup, most baselines failed to preserve their previous performance. MLP had the largest decline, with F1 decreasing from 0.620 to 0.290. Even the more stable tree-based models followed the same downward direction and failed to gain performance in the new stage. This consistency suggests that the problem is not limited to a particular algorithm or an unstable run. Rather, it reflects a broader difficulty for hidden-parameter and tree-structure models in carrying forward useful knowledge when the feature set itself changes.

MHL showed a qualitatively different pattern. It was the only method whose performance did not decline from Stage 1 to Stage 2. Instead, its F1 increased from 0.668 to 0.683. This result is consistent with the workflow described earlier in Figure 1. MHL does not need to overwrite invisible parameters to adapt. It starts from a previously validated white-box rule base, explicitly identifies obsolete features such as SIRS, and incorporates new signals such as SOFA through

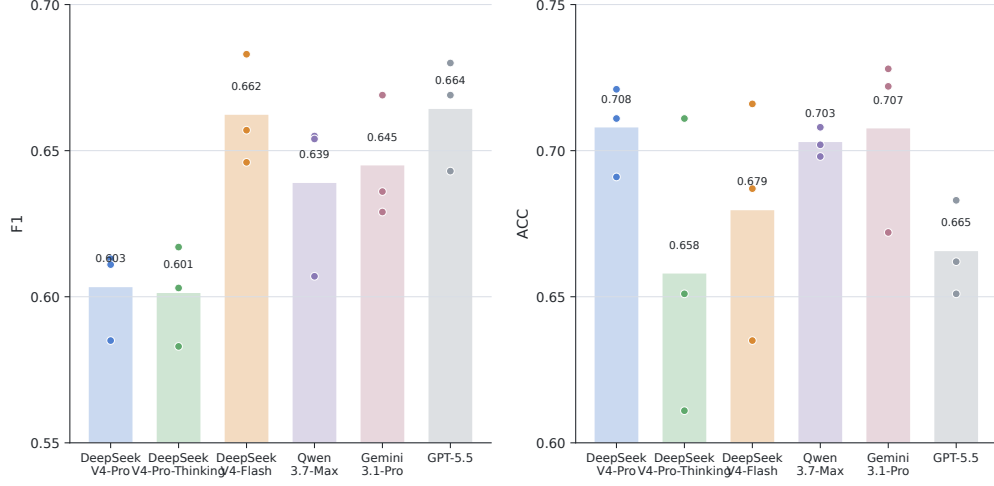


Figure 7: Comparison of different LLM backends on UKB (Left) and CCID (Right).

code-level rule revision. In this sense, the continual learning result supports the central claim of the paper. Explicit rule revision provides a more auditable and potentially safer route for adapting medical tabular models when feature evolution would otherwise induce catastrophic forgetting.

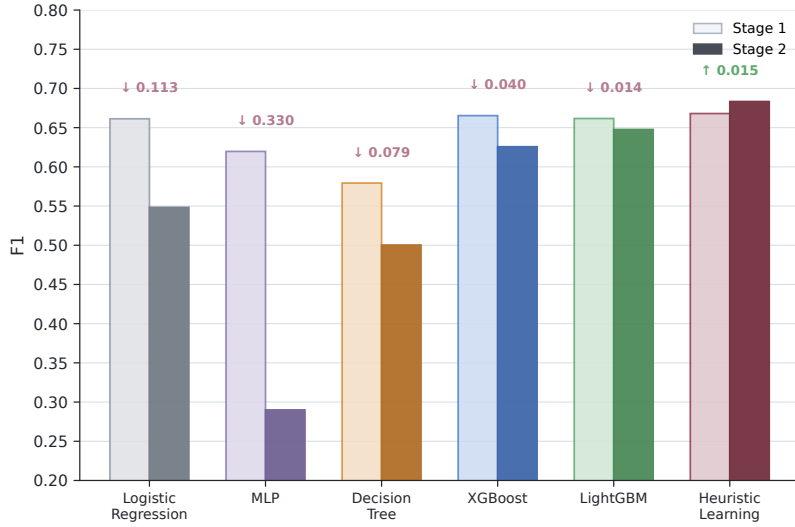


Figure 8: Continual learning performance on MIMIC across two stages.

3 Discussion

The experiments show that Medical Heuristic Learning achieves performance comparable to state-of-the-art tabular prediction models while preserving a fully white-box, interpretable, and auditable model form. This finding is important because it challenges a common assumption in clinical prediction. High-performing systems are often assumed to require black-box models, whereas interpretable systems are often expected to sacrifice expressive power. The advantage of MHL is most apparent in the settings that are often most difficult in practice, namely small training samples and severely imbalanced class distributions. Under these conditions, strong baselines such as XGBoost [16], LightGBM [17], FT-Transformer [19], and MLP [18] were more prone to performance degradation or one-sided prediction behavior, whereas MHL retained a more usable predictive profile. In larger and more balanced training settings, conventional tabular models remained highly competitive and in some cases achieved stronger numerical results. Even so, MHL reached a similar performance range while maintaining a substantially more interpretable model

form. Its value therefore lies not only in predictive accuracy, but also in the fact that the final model remains readable, executable, and open to direct inspection.

This behavior is closely related to the constrained workflow in which MHL operates. Statistical probes, medical knowledge probes, validation feedback, and degradation analysis jointly guide rule synthesis and rule revision. Such a design is particularly relevant in low-sample and imbalanced regimes, where models driven primarily by gradient optimization or greedy partitioning may overfit accidental correlations or become dominated by majority-class signals. MHL addresses this problem by combining multiple forms of constraint. The statistical probe anchors rule generation in observed distributional patterns, whereas the medical knowledge probe introduces clinical semantics, candidate thresholds, and domain priors. The ablation results support the complementarity of these two information sources. Once these signals are encoded as executable rules, the decision boundary becomes explicit rather than remaining hidden in parameter space, making bias, failure modes, and local errors easier to identify and revise. Stability is further strengthened by versioned rule refinement, in which updates are introduced through targeted modifications informed by errors and degradations rather than through wholesale rewriting. Taken together, these findings suggest that the robustness of MHL arises not from any single component, but from the interaction among statistical evidence, medical priors, deterministic rules, and explicit iterative revision.

The interpretability offered by MHL also differs from both post-hoc explanation methods and the transparency typically associated with simple models. SHAP and LIME are widely used to explain black-box models by approximating model behavior after training [2, 3]. Although useful, these explanations are not the executable logic of the model itself and may depend on background distributions, perturbation strategies, and modeling assumptions [4, 5]. Linear models and shallow decision trees, by contrast, are easier to inspect because of their simpler structures, but structural simplicity alone does not provide a record of how medical knowledge entered the model, how errors were analyzed, or how successive versions were revised. MHL provides a different form of interpretability. The final predictor is executable rule code accompanied by statistical evidence, medical rationale, error analysis, degradation warnings, versioned edits, and final selection records. What emerges is not merely an explanation of an isolated output, but an auditable trajectory of model formation and refinement. In practical clinical use, clinicians can directly examine the medical plausibility of the generated rules, and the explicit rule structure may even suggest candidate disease patterns that warrant further clinical investigation. This does not imply that every rule is causally correct or clinically validated. Rather, it means that the model’s reasoning and revision process is exposed to scrutiny in a way that conventional black-box systems rarely allow.

The continual learning experiment extends this argument from static prediction to model governance under feature evolution. Clinical data environments are inherently dynamic. Measurements, scoring systems, disease definitions, practice guidelines, and feature distributions may all change as medical knowledge and care delivery evolve. For conventional models, feature removal, feature addition, and distributional shift together create a difficult adaptation problem. Even when a model can be retrained or transferred, it is often unclear how prior knowledge has been preserved, how newly introduced variables alter decision logic, or which parts of the previous model have effectively been overwritten. MHL offers a more transparent adaptation pathway. It begins from the previous rule base, explicitly records removed, added, and retained features, and incorporates new evidence through renewed probes and code-level rule revision. This mechanism helps explain why MHL can retain useful performance under feature evolution, because adaptation is guided simultaneously by previously validated logic and by newly observed statistical and clinical signals. At the same time, the adapted model remains interpretable because changes appear as visible modifications to rule code rather than as hidden shifts in parameters. In high-stakes clinical settings, this distinction matters because model updating is not only a question of predictive performance, but also one of version control, accountability, and risk review.

Several boundaries of MHL should also be made explicit. First, the present evaluation focused on binary prediction tasks using clinical tabular data. Whether the same workflow can be extended to imaging, free text, multimodal prediction, multiclass outcomes, survival analysis, or complex longitudinal modeling remains to be established. Second, explicit rule systems may face scalability challenges when the feature space becomes very high-dimensional or when clinically relevant interactions are highly complex. In such settings, rule combinations may become difficult to read, maintain, or validate. Third, although MHL does not use the LLM as a direct classifier, the choice of LLM backend can still influence candidate rules, decision style, and revision trajectories, as indicated by the backend comparison experiment. Finally, the evidence reported here is based on offline evaluation, retrospective datasets, and a simulated feature evolution setting. These findings therefore cannot substitute for prospective, multicenter validation in real clinical workflows. White-box audibility increases the possibility of review, but it does not in itself guarantee causal validity, clinical safety, or readiness for deployment.

Overall, MHL suggests an LLM-based alternative to conventional black-box learning for clinical prediction, one that integrates medical priors, data evidence, executable rules, and explicit updating within a unified white-box framework.

Its significance is not limited to interpretability alone. More broadly, it addresses several requirements that are central to high-risk digital medicine, including competitive performance, auditability, and the capacity to adapt as clinical data structures evolve. Future work should evaluate this framework across more complex tasks, external cohorts, and prospective clinical settings, while also developing principled methods for controlling rule complexity and governing rule updates over time. In the longer term, LLMs should remain assistive components that help generate auditable rule candidates rather than substitutes for clinical or statistical judgment. The broader implication of this study is that, in medical tabular prediction, performance, interpretability, auditability, and adaptability need not be treated as mutually exclusive goals. They can be pursued together within a white-box learning framework designed for clinical accountability.

4 Methods

This section describes the MHL framework and the experimental setup used in this study. Section 4.1 presents the heuristic learning workflow, and Section 4.2 summarizes the experimental configuration.

4.1 Medical Heuristic Learning (MHL) Framework

MHL is a constrained white-box learning workflow composed of a statistical probe, a medical knowledge probe, initial rule generation, iterative rule refinement, and continual learning under feature evolution. As shown in Figure 1, subfigure (a) summarizes the standard MHL workflow, whereas subfigure (b) summarizes its continual learning extension under feature evolution. The standard workflow is conditioned not only on tabular data and feature descriptions, but also on a task description and a metric priority description. The main components are described below.

4.1.1 Statistical Probe

The statistical probe extracts structured empirical signals from the training set. Its inputs are the full training table, the target label, and feature-type information. Its output is a ranked CSV artifact, whose exported fields are summarized in Table 1.

Taken together, these fields organize the probe output into a structured empirical summary for downstream rule construction. In particular, `rank` and `feature` serve as indexing fields that identify and order candidate variables, whereas the remaining fields summarize distributional properties and univariate association signals. Figure 2 provides a visual example of this artifact format.

4.1.2 Medical Knowledge Probe

The medical knowledge probe transforms empirical feature signals into clinically grounded rule design context. Its inputs are the candidate features, their univariate summaries, and the accompanying feature descriptions. Rather than returning free-form commentary, it is required to produce an English Markdown table with fixed columns, as summarized in Table 2. The corresponding standard knowledge probe prompt is provided in Appendix B.1.

This design ensures that the generated rules are grounded not only in statistical association, but also in clinically meaningful rationale. More broadly, the table links empirical signals to interpretable clinical explanations and directly usable cues for downstream rule construction. Figure 2 shows how this artifact appears in practice.

4.1.3 Initial Rule Generation

After the two probes have produced structured evidence, MHL enters the initial rule generation stage. At this stage, the LLM receives the outputs of the statistical and medical knowledge probes together with the metric priority description and the task description, and is asked to synthesize an initial executable classifier. It is not allowed to return natural-language advice alone. Instead, it must return strict JSON containing three fields: `version`, `error_analysis`, and `new_policy_code`. The `version` field is fixed to `v0`, `error_analysis` contains the design rationale in English, and `new_policy_code` must contain a complete Python function definition named `predict_v0`. The corresponding standard initial rule generation prompt is provided in Appendix B.2.

The generated function is constrained in several ways. Its signature must be `def predict_v0(features: dict) -> int:`, and it must return an integer label 0 or 1 consistent with the dataset encoding. The rule must be deterministic, self-contained, and restricted to the Python standard library. Each `if`, `elif`, or `else` branch must include a brief English comment explaining the medical rationale or design intent. Together, these constraints make the initial rule both executable and auditable.

Table 1: Fields exported by the statistical probe artifact.

Category	Variable type	Field	Interpretation
Descriptive statistics	Continuous and discrete	rank	Relevance-based rank order of the candidate feature in the exported probe table.
		feature	Feature name.
		feature_type	Variable type assigned by the probe, such as continuous or binary.
		n_valid	Number of non-missing observations used for the feature.
	Continuous	missing_rate	Proportion of missing values for the feature.
		mean	Sample mean.
		std	Sample standard deviation.
		median	Sample median.
		min	Minimum observed value.
		max	Maximum observed value.
		binned_or_q4_rel_to_q1	Optional quantile-based risk summary, such as the relative risk of higher bins versus the first quartile.
	Discrete	n_unique	Number of unique levels.
		level_counts	Per-level counts, typically stored as a serialized dictionary.
Univariate analysis	Continuous and discrete	method	Statistical method selected for the feature, such as pointbiserial, mwu, or chi2.
		statistic	Main test statistic returned by the selected method.
		p_value	Main p-value returned by the selected method.
	Continuous	direction	Direction of association when applicable.
		pointbiserial_r	Point-biserial correlation coefficient for continuous features in binary tasks.
		pointbiserial_p	P-value associated with the point-biserial correlation.
		mwu_u	Mann-Whitney U statistic.
		mwu_p	P-value associated with the Mann-Whitney U test.
	Discrete	chi2_stat	Chi-square statistic for discrete features.
		chi2_p	P-value associated with the chi-square test.

Table 2: Fixed output columns required from the medical knowledge probe.

Field	Function in downstream rule generation
Feature	Identifies the candidate variable to be referenced in rule code.
Univariate signal (summary)	Provides a short natural-language summary of the empirical association, such as strong positive, weak to moderate, or mixed association.
Clinical rationale	Provides concise medical justification, including typical mechanisms, clinical context, or caveats when relevant.
Suggested threshold	Supplies a directly usable rule condition, which may be a numeric threshold or a categorical trigger such as Present (1).
Evidence (high/medium/low)	confidence Indicates the expected confidence level of the clinical prior during rule construction.

4.1.4 Rule Iteration

The rule iteration procedure is summarized by the swimlane diagram in Figure 9. The purpose of this stage is not to rewrite the model from scratch at each round, but to introduce small, evidence-based, versioned changes to the current rule. The workflow unfolds through two interacting lanes, with data feedback and analysis on the left and rule evolution and management on the right. This structure is central to the method because it makes each modification attributable to explicit evidence rather than to opaque parameter updates.

The iteration cycle begins with the current rule version, denoted V_t , which is executed on the training set to produce case-level predictions. The system then collects error samples, including misclassified cases and their false-positive and false-negative counts. It next identifies degradation samples, namely cases that were correctly predicted by the previous version but became incorrect after the most recent change, and constructs a degradation warning from these regressions. In parallel, the workflow retrieves the iteration history, including previous version records and prior modification rationales. Together, these artifacts provide a structured account of what the current rule gets wrong, what has recently deteriorated, and what has already been attempted.

These materials are then assembled into an iteration context that includes the full current code, the training error analysis, the version trajectory, any degradation warning, and the metric priority description. This context is passed to the LLM under a strict minimal-change constraint. The model is instructed to prioritize regressed cases when degradation exists, avoid collapsing into near-constant prediction, and modify only a small number of thresholds, weights, or rules in any single round. The output format remains strict JSON and again contains `version`, `error_analysis`, and `new_policy_code`. The new code must preserve the same execution and commenting conventions as in the initial rule generation stage. In this sense, the LLM functions as a controlled code editor rather than as an unconstrained generator. The corresponding standard iteration prompt is provided in Appendix B.3.

After a candidate rule V_{t+1} is produced, the workflow enters a validation and management phase. The returned JSON is parsed, the generated Python code undergoes syntax checking, and the function naming convention is verified. The candidate is then evaluated on the designated evaluation split, after which the new version and its iteration note are appended to the rule history together with the corresponding metrics. The outcome of this stage is therefore not only a selected rule, but also a versioned revision record that preserves the rationale for each accepted modification.

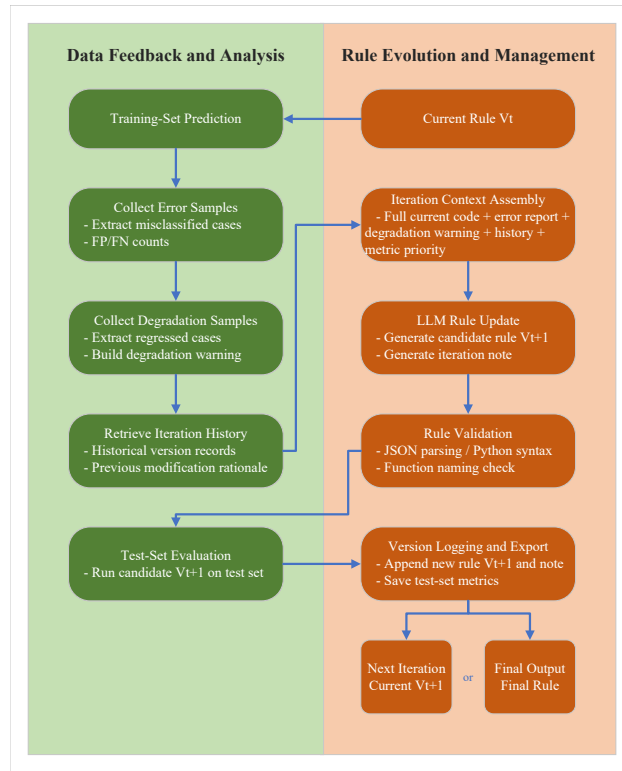


Figure 9: Detailed swimlane view of the rule iteration stage in MHL, showing the closed-loop process of error analysis, degradation detection, proposal generation, rule validation, and version acceptance.

4.1.5 Continual Learning

MHL extends the same white-box logic to feature evolution. As shown in Figure 1, the continual learning pipeline in subfigure (b) is not a separate method detached from the standard workflow in subfigure (a). It preserves the same core sequence of statistical probing, medical knowledge probing, initial rule generation, and iterative rule refinement. The key difference is that continual learning begins from previously validated artifacts rather than from an empty state.

Specifically, the new stage uses the previous probe artifacts and the previously accepted rule as explicit adaptation context, while still conditioning rule generation on the updated task and metric descriptions.

When the feature space changes, the workflow first determines which columns have been removed, added, or renamed, and then refreshes the outputs of the univariate and medical knowledge probes accordingly. In this adapted setting, the v0 generation stage is conditioned not only on the updated probe evidence and the metric priority description, but also on the final rule accepted in the previous stage, which serves as an explicit blueprint for revision. The knowledge probe template itself remains unchanged from standard MHL; however, during continual learning context assembly, it is instantiated with only the newly added columns rather than the full feature set. By contrast, the continual v0 generation prompt introduces an explicit summary of feature drift, and its full template is reproduced in Appendix B.4. Once this adapted v0 rule has been produced, subsequent refinement proceeds with the same iteration prompt used in standard MHL. Continual learning in MHL is therefore realized through explicit inheritance of prior white-box artifacts together with explicit revision under updated evidence, making both continuity and change across stages transparent and auditable.

4.2 Experimental Setup

4.2.1 Datasets, Preprocessing, and Data Splitting

We evaluated MHL on three medical tabular datasets. UK Biobank (UKB) is a large public biomedical cohort resource, and in this study it was used for depression prediction from blood metabolomic measurements [11]. The Critical Care Information Database (CCID) is a private clinical dataset and serves as the in-hospital critical care prediction benchmark in this work [13]. Because access to UKB requires paid subscription and a relatively complex application process, and because the open-release plan for CCID is still in progress, the LLMs used in our experiments had not been trained on either UKB or CCID data. The Medical Information Mart for Intensive Care (MIMIC) is a public intensive care database [12], and in this study it was used primarily for the continual learning experiment under feature evolution. Across datasets, the tasks remained binary prediction problems, although the predictor variables and clinical settings differed by dataset source. The exact feature lists used in each dataset are provided in the supplementary materials.

For data handling, missing values were imputed with feature means. Additional normalization was applied before training for Logistic Regression [14], MLP [18], and FT-Transformer [19] because these models are more sensitive to feature scale. The training, validation, and test sets were generated by one-time random sampling and then held fixed within each experimental configuration so that all methods were compared on identical splits. Different experiments then modified one factor at a time within this fixed-split design, such as training set size, class ratio, backend model, or stage-specific feature space.

4.2.2 Baselines

The baseline set was chosen to cover the main methodological families used in clinical tabular prediction. It included Logistic Regression [14] as a linear model, Decision Tree [15] as an interpretable tree-based learner, XGBoost [16] and LightGBM [17] as strong gradient-boosted tree ensembles, MLP [18] as a standard neural baseline, and FT-Transformer [19] as a modern deep tabular model. This design ensured that MHL was compared not with a single competing paradigm, but with representative methods spanning linear, tree-based, ensemble, and deep-learning approaches to tabular prediction.

4.2.3 Evaluation Metrics

All methods were evaluated using ACC, F1, Sensitivity, and Specificity. ACC captures overall correctness across all samples. F1 summarizes the trade-off between false positives and false negatives in a single score and is particularly informative when class distributions are skewed. Sensitivity measures the ability to detect positive cases, whereas Specificity measures the ability to avoid false positives among negative cases. Taken together, these four metrics provide a concise but clinically meaningful view of performance under balanced, small-sample, imbalanced, and feature evolution settings.

4.2.4 Environment and Parameters

All experiments were run under Linux with Python 3.11. Unless otherwise specified, experiments were repeated with random seeds 36, 40, and 42. Across most experiments, the underlying LLM backend was fixed to DeepSeek-V4-Pro, and the probe generation, rule synthesis, and iterative refinement procedures were kept unchanged. Only the backend comparison experiment varied the foundation model itself. Other experimental manipulations, such as training set

size, class ratio, and stage-specific feature space, are described in Results, while the same overall MHL workflow was preserved across settings.

Code Availability

The code for Medical Heuristic Learning is publicly available at <https://github.com/MPU-Li-OmicsLab/medical-heuristic-learning>.

Data Availability

Three datasets were used in this study: UK Biobank (UKB), the Critical Care Information Database (CCID), and the Medical Information Mart for Intensive Care (MIMIC).

The UKB data and MIMIC data are subject to user agreement restrictions and therefore cannot be publicly shared by the authors.

The CCID data are available from <http://ccid.sdytyhdy.cn:60011/>. In addition, a small subset of CCID data required for running the simplest example code is directly provided in the GitHub repository.

Acknowledgments

This work is supported by Macao Polytechnic University (RP/FCA-14/2023), the Macao Science and Technology Development Fund (FDCT, 0033/2023/RIB2, 0062/2025/RIB2), Joint Fund between FDCT and the Department of Science and Technology of Guangdong Province (FDCT-GDST, 0009/2024/AGJ), and Joint Fund between FDCT and the Ministry of Science and Technology of China (FDCT-MOST, 0106/2025/AMJ). The Macao Polytechnic University submission approval ID is fca.f673.ffe3.8.

The co-corresponding author in this study had obtained certification through the Collaborative Institutional Training Initiative (CITI program) for international cooperation and training. This research has been conducted using the UK Biobank Resource under Application Number 99946. The study protocol has been reviewed and approved by the Institutional Review Board (IRB) of Yantai Yuhuangding Hospital (file number: 2024-018).

References

- [1] Shuroug A. Alowais, Sahar S. Alghamdi, Nada Alsuhebany, Tariq Alqahtani, Abdulrahman I. Alshaya, Sumaya N. Almohareb, Atheer Aldairem, Mohammed Alrashed, Khalid Bin Saleh, Hisham A. Badreldin, Majed S. Al Yami, Shmeylan Al Harbi, and Abdulkareem M. Albekairy. Revolutionizing healthcare: the role of artificial intelligence in clinical practice. *BMC Medical Education*, 23(1), sep 2023.
- [2] Scott M. Lundberg and Su-In Lee. A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*, pages 4765–4774, 2017.
- [3] Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin. "why should i trust you?": Explaining the predictions of any classifier. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 1135–1144. Association for Computing Machinery, 2016.
- [4] Cynthia Rudin. Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead. *Nature Machine Intelligence*, 1(5):206–215, may 2019.
- [5] Erico Tjoa and Cuntai Guan. A survey on explainable artificial intelligence (xai): Toward medical xai. *IEEE Transactions on Neural Networks and Learning Systems*, 32(11):4793–4813, nov 2021.
- [6] Christopher J. Kelly, Alan Karthikesalingam, Mustafa Suleyman, Greg Corrado, and Dominic King. Key challenges for delivering clinical impact with artificial intelligence. *BMC Medicine*, 17(1), oct 2019.
- [7] Mervyn Singer, Clifford S. Deutschman, Christopher Warren Seymour, Manu Shankar-Hari, Djillali Annane, Michael Bauer, Rinaldo Bellomo, Gordon R. Bernard, Jean-Daniel Chiche, Craig M. Coopersmith, Richard S. Hotchkiss, Mitchell M. Levy, John C. Marshall, Greg S. Martin, Steven M. Opal, Gordon D. Rubenfeld, Tom van der Poll, Jean-Louis Vincent, and Derek C. Angus. The third international consensus definitions for sepsis and septic shock (sepsis-3). *JAMA*, 315(8):801–810, feb 2016.
- [8] James Kennedy and Russell C. Eberhart. Particle swarm optimization. In *Proceedings of the IEEE International Conference on Neural Networks (ICNN 1995)*, volume 4, pages 1942–1948. IEEE, nov 1995.

- [9] Marco Dorigo, Vittorio Maniezzo, and Alberto Colorni. Ant system: Optimization by a colony of cooperating agents. *IEEE Transactions on Systems, Man, and Cybernetics, Part B: Cybernetics*, 26(1):29–41, feb 1996.
- [10] Jiayi Weng. Learning beyond gradients. <https://trinkle23897.github.io/learning-beyond-gradients/>, May 2026. Blog post.
- [11] Cathie Sudlow, John Gallacher, Naomi Allen, Valerie Beral, Paul Burton, John Danesh, Paul Downey, Paul Elliott, Jane Green, Martin Landray, Bette Liu, Paul Matthews, Ken Ong, Jill Pell, Alan Silman, Alan Young, Tim Sprosen, Tim Peakman, and Rory Collins. Uk biobank: an open access resource for identifying the causes of a wide range of complex diseases of middle and old age. *PLoS Medicine*, 12(3):e1001779, 2015.
- [12] Alistair E. W. Johnson, Tom J. Pollard, Lu Shen, Li-wei H. Lehman, Mengling Feng, Mohammad Ghassemi, Benjamin Moody, Peter Szolovits, Leo Anthony Celi, and Roger G. Mark. Mimic-iii, a freely accessible critical care database. *Scientific Data*, 3:160035, 2016.
- [13] Jing Wang, Qixiu Li, Can Xie, Xiaofei Li, Huikao Wang, Wei Xu, Ruyan Lv, Xiaobing Zhai, Ping Xu, Kefeng Li, and Xi-Cheng Song. Predicting in-hospital mortality in intensive care unit patients using causal survivalnet with serum chloride and other causal factors: cross-country study. *Journal of Medical Internet Research*, 27:e70118, jul 2025.
- [14] David R. Cox. The regression analysis of binary sequences. *Journal of the Royal Statistical Society: Series B (Methodological)*, 20(2):215–232, 1958.
- [15] Leo Breiman, Jerome H. Friedman, Richard A. Olshen, and Charles J. Stone. *Classification and Regression Trees*. Wadsworth International Group, 1984.
- [16] Tianqi Chen and Carlos Guestrin. Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794. Association for Computing Machinery, 2016.
- [17] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. Lightgbm: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*, pages 3146–3154, 2017.
- [18] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323(6088):533–536, oct 1986.
- [19] Yury Gorishniy, Ivan Rubachev, Valentin Khrulkov, and Artem Babenko. Revisiting deep learning models for tabular data. In *Advances in Neural Information Processing Systems 34 (NeurIPS 2021)*, pages 18932–18943, 2021.

A Supplementary Tables

Table 3: Ablation summary across probe settings.

Dataset	Probe	Train Size	ACC	F1	Sensitivity	Specificity
UKB	N	10	0.503	0.623	0.837	0.169
UKB	N	100	0.511	0.547	0.736	0.286
UKB	N	1000	0.531	0.528	0.522	0.540
UKB	N	3000	0.525	0.547	0.629	0.420
UKB	K	10	0.556	0.602	0.683	0.429
UKB	K	100	0.529	0.654	0.891	0.167
UKB	K	1000	0.515	0.627	0.823	0.207
UKB	K	3000	0.547	0.583	0.643	0.451
UKB	S	10	0.526	0.524	0.533	0.520
UKB	S	100	0.561	0.575	0.597	0.525
UKB	S	1000	0.573	0.643	0.769	0.377
UKB	S	3000	0.585	0.613	0.657	0.513
UKB	S+K	10	0.533	0.623	0.781	0.285
UKB	S+K	100	0.548	0.587	0.645	0.451
UKB	S+K	1000	0.583	0.603	0.635	0.531
UKB	S+K	3000	0.561	0.621	0.723	0.399
CCID	N	10	0.570	0.639	0.765	0.375
CCID	N	100	0.667	0.671	0.676	0.658
CCID	N	1000	0.558	0.682	0.948	0.168
CCID	N	3000	0.554	0.677	0.931	0.177
CCID	K	10	0.583	0.685	0.907	0.259
CCID	K	100	0.618	0.681	0.819	0.418
CCID	K	1000	0.594	0.689	0.900	0.287
CCID	K	3000	0.629	0.693	0.832	0.427
CCID	S	10	0.571	0.601	0.660	0.482
CCID	S	100	0.665	0.707	0.811	0.518
CCID	S	1000	0.707	0.675	0.627	0.787
CCID	S	3000	0.712	0.716	0.726	0.699
CCID	S+K	10	0.641	0.685	0.786	0.497
CCID	S+K	100	0.700	0.710	0.735	0.665
CCID	S+K	1000	0.708	0.728	0.783	0.632
CCID	S+K	3000	0.705	0.727	0.785	0.625

Table 4: Summary across training set sizes.

Dataset	Model	Train Size	ACC	F1	Sensitivity	Specificity
UKB	Logistic Regression	10	0.555	0.520	0.506	0.604
UKB	Logistic Regression	50	0.524	0.480	0.441	0.607
UKB	Logistic Regression	100	0.559	0.567	0.578	0.540
UKB	Logistic Regression	500	0.596	0.597	0.600	0.591
UKB	Logistic Regression	1000	0.619	0.619	0.621	0.616
UKB	Logistic Regression	3000	0.633	0.625	0.613	0.652
UKB	MLP	10	0.547	0.498	0.483	0.611
UKB	MLP	50	0.547	0.483	0.429	0.665
UKB	MLP	100	0.535	0.568	0.612	0.457
UKB	MLP	500	0.592	0.598	0.609	0.575
UKB	MLP	1000	0.594	0.610	0.635	0.552
UKB	MLP	3000	0.633	0.633	0.632	0.635
UKB	Decision Tree	10	0.524	0.512	0.507	0.540

Continued on next page

Continued from previous page

Dataset	Model	Train Size	ACC	F1	Sensitivity	Specificity
UKB	Decision Tree	50	0.537	0.506	0.480	0.594
UKB	Decision Tree	100	0.525	0.537	0.553	0.496
UKB	Decision Tree	500	0.543	0.544	0.545	0.541
UKB	Decision Tree	1000	0.542	0.539	0.536	0.549
UKB	Decision Tree	3000	0.591	0.594	0.599	0.584
UKB	XGBoost	10	0.529	0.523	0.531	0.527
UKB	XGBoost	50	0.567	0.552	0.534	0.601
UKB	XGBoost	100	0.577	0.580	0.586	0.569
UKB	XGBoost	500	0.620	0.624	0.629	0.612
UKB	XGBoost	1000	0.633	0.633	0.633	0.633
UKB	XGBoost	3000	0.671	0.669	0.667	0.675
UKB	LightGBM	10	0.500	0	0	1
UKB	LightGBM	50	0.548	0.525	0.501	0.595
UKB	LightGBM	100	0.562	0.558	0.556	0.567
UKB	LightGBM	500	0.605	0.612	0.622	0.589
UKB	LightGBM	1000	0.634	0.633	0.633	0.635
UKB	LightGBM	3000	0.688	0.688	0.687	0.689
UKB	FT-Transformer	10	0.500	0.667	1	0
UKB	FT-Transformer	50	0.500	0.667	1	0
UKB	FT-Transformer	100	0.500	0.667	1	0
UKB	FT-Transformer	500	0.500	0.667	1	0
UKB	FT-Transformer	1000	0.500	0.667	1	0
UKB	FT-Transformer	3000	0.500	0.667	1	0
UKB	MHL	10	0.533	0.623	0.781	0.285
UKB	MHL	50	0.534	0.597	0.716	0.351
UKB	MHL	100	0.548	0.587	0.645	0.451
UKB	MHL	500	0.537	0.620	0.765	0.308
UKB	MHL	1000	0.583	0.603	0.635	0.531
UKB	MHL	3000	0.561	0.621	0.723	0.399
CCID	Logistic Regression	10	0.573	0.531	0.495	0.651
CCID	Logistic Regression	50	0.671	0.655	0.627	0.715
CCID	Logistic Regression	100	0.704	0.702	0.698	0.711
CCID	Logistic Regression	500	0.746	0.735	0.703	0.789
CCID	Logistic Regression	1000	0.752	0.736	0.694	0.809
CCID	Logistic Regression	3000	0.758	0.738	0.681	0.835
CCID	MLP	10	0.565	0.525	0.490	0.640
CCID	MLP	50	0.662	0.640	0.602	0.723
CCID	MLP	100	0.673	0.676	0.686	0.660
CCID	MLP	500	0.699	0.724	0.785	0.614
CCID	MLP	1000	0.757	0.745	0.712	0.801
CCID	MLP	3000	0.720	0.673	0.579	0.860
CCID	Decision Tree	10	0.537	0.471	0.428	0.646
CCID	Decision Tree	50	0.644	0.625	0.603	0.684
CCID	Decision Tree	100	0.667	0.672	0.678	0.657
CCID	Decision Tree	500	0.657	0.661	0.669	0.646
CCID	Decision Tree	1000	0.674	0.680	0.695	0.653
CCID	Decision Tree	3000	0.657	0.600	0.515	0.800
CCID	XGBoost	10	0.519	0.494	0.489	0.548
CCID	XGBoost	50	0.666	0.660	0.654	0.678
CCID	XGBoost	100	0.697	0.693	0.688	0.705
CCID	XGBoost	500	0.746	0.748	0.753	0.739
CCID	XGBoost	1000	0.769	0.770	0.772	0.767
CCID	XGBoost	3000	0.752	0.711	0.611	0.893
CCID	LightGBM	10	0.500	0	0	1
CCID	LightGBM	50	0.603	0.603	0.605	0.601

Continued on next page

Continued from previous page

Dataset	Model	Train Size	ACC	F1	Sensitivity	Specificity
CCID	LightGBM	100	0.685	0.678	0.667	0.702
CCID	LightGBM	500	0.740	0.741	0.745	0.736
CCID	LightGBM	1000	0.762	0.758	0.749	0.775
CCID	LightGBM	3000	0.749	0.704	0.599	0.899
CCID	FT-Transformer	10	0.509	0.624	0.825	0.193
CCID	FT-Transformer	50	0.522	0.665	0.946	0.099
CCID	FT-Transformer	100	0.501	0.664	0.988	0.013
CCID	FT-Transformer	500	0.515	0.664	0.959	0.071
CCID	FT-Transformer	1000	0.632	0.701	0.850	0.414
CCID	FT-Transformer	3000	0.697	0.734	0.839	0.555
CCID	MHL	10	0.641	0.685	0.786	0.497
CCID	MHL	50	0.680	0.709	0.780	0.579
CCID	MHL	100	0.700	0.710	0.735	0.665
CCID	MHL	500	0.608	0.690	0.871	0.344
CCID	MHL	1000	0.708	0.728	0.783	0.632
CCID	MHL	3000	0.705	0.727	0.785	0.625

Table 5: Summary under different positive-to-negative ratios.

Dataset	Train Size	Model	Pos:Neg	ACC	F1	Sensitivity	Specificity
UKB	1000	Logistic Regression	50:1	0.500	0.666	0.999	0.001
UKB	1000	Logistic Regression	10:1	0.501	0.666	0.996	0.006
UKB	1000	Logistic Regression	5:1	0.508	0.666	0.983	0.032
UKB	1000	Logistic Regression	2:1	0.568	0.671	0.881	0.256
UKB	1000	Logistic Regression	1:1	0.618	0.611	0.601	0.634
UKB	1000	Logistic Regression	1:2	0.566	0.357	0.243	0.888
UKB	1000	Logistic Regression	1:5	0.527	0.126	0.069	0.985
UKB	1000	Logistic Regression	1:10	0.513	0.065	0.034	0.991
UKB	1000	Logistic Regression	1:50	0.502	0.013	0.007	0.997
UKB	1000	MLP	50:1	0.500	0.667	1	0
UKB	1000	MLP	10:1	0.500	0.667	1	0
UKB	1000	MLP	5:1	0.501	0.667	0.998	0.004
UKB	1000	MLP	2:1	0.544	0.676	0.951	0.137
UKB	1000	MLP	1:1	0.602	0.609	0.621	0.583
UKB	1000	MLP	1:2	0.554	0.319	0.229	0.878
UKB	1000	MLP	1:5	0.504	0.021	0.011	0.998
UKB	1000	MLP	1:10	0.500	0	0	1
UKB	1000	MLP	1:50	0.500	0	0	1
UKB	1000	Decision Tree	50:1	0.509	0.663	0.967	0.051
UKB	1000	Decision Tree	10:1	0.516	0.648	0.890	0.142
UKB	1000	Decision Tree	5:1	0.528	0.635	0.823	0.234
UKB	1000	Decision Tree	2:1	0.564	0.613	0.693	0.435
UKB	1000	Decision Tree	1:1	0.562	0.562	0.564	0.559
UKB	1000	Decision Tree	1:2	0.550	0.473	0.404	0.695
UKB	1000	Decision Tree	1:5	0.546	0.369	0.267	0.825
UKB	1000	Decision Tree	1:10	0.531	0.271	0.174	0.889
UKB	1000	Decision Tree	1:50	0.507	0.074	0.039	0.974
UKB	1000	XGBoost	50:1	0.500	0.667	1	0
UKB	1000	XGBoost	10:1	0.507	0.669	0.995	0.018
UKB	1000	XGBoost	5:1	0.526	0.676	0.989	0.064
UKB	1000	XGBoost	2:1	0.596	0.684	0.876	0.316
UKB	1000	XGBoost	1:1	0.631	0.626	0.617	0.645
UKB	1000	XGBoost	1:2	0.600	0.455	0.334	0.865

Continued on next page

Continued from previous page

Dataset	Train Size	Model	Pos:Neg	ACC	F1	Sensitivity	Specificity
UKB	1000	XGBoost	1:5	0.519	0.109	0.059	0.979
UKB	1000	XGBoost	1:10	0.505	0.025	0.013	0.997
UKB	1000	XGBoost	1:50	0.500	0.001	0.001	1
UKB	1000	LightGBM	50:1	0.500	0.667	1	0
UKB	1000	LightGBM	10:1	0.504	0.668	0.997	0.011
UKB	1000	LightGBM	5:1	0.520	0.672	0.985	0.056
UKB	1000	LightGBM	2:1	0.596	0.684	0.874	0.319
UKB	1000	LightGBM	1:1	0.637	0.637	0.636	0.639
UKB	1000	LightGBM	1:2	0.608	0.472	0.351	0.864
UKB	1000	LightGBM	1:5	0.515	0.097	0.052	0.978
UKB	1000	LightGBM	1:10	0.506	0.028	0.014	0.999
UKB	1000	LightGBM	1:50	0.500	0	0	1
UKB	1000	FT-Transformer	50:1	0.500	0.667	1	0
UKB	1000	FT-Transformer	10:1	0.500	0.667	1	0
UKB	1000	FT-Transformer	5:1	0.500	0.667	1	0
UKB	1000	FT-Transformer	2:1	0.500	0.667	1	0
UKB	1000	FT-Transformer	1:1	0.500	0.667	1	0
UKB	1000	FT-Transformer	1:2	0.500	0	0	1
UKB	1000	FT-Transformer	1:5	0.500	0	0	1
UKB	1000	FT-Transformer	1:10	0.500	0	0	1
UKB	1000	FT-Transformer	1:50	0.500	0	0	1
UKB	1000	MHL	50:1	0.519	0.573	0.674	0.365
UKB	1000	MHL	10:1	0.567	0.656	0.827	0.308
UKB	1000	MHL	5:1	0.561	0.661	0.851	0.272
UKB	1000	MHL	2:1	0.558	0.639	0.785	0.331
UKB	1000	MHL	1:1	0.583	0.603	0.635	0.531
UKB	1000	MHL	1:2	0.526	0.598	0.722	0.329
UKB	1000	MHL	1:5	0.561	0.579	0.640	0.482
UKB	1000	MHL	1:10	0.528	0.428	0.403	0.654
UKB	1000	MHL	1:50	0.563	0.442	0.377	0.749
UKB	3000	Logistic Regression	50:1	0.500	0.667	1	0
UKB	3000	Logistic Regression	10:1	0.500	0.667	1	0
UKB	3000	Logistic Regression	5:1	0.505	0.668	0.996	0.014
UKB	3000	Logistic Regression	2:1	0.571	0.678	0.903	0.239
UKB	3000	Logistic Regression	1:1	0.626	0.621	0.611	0.641
UKB	3000	Logistic Regression	1:2	0.580	0.361	0.237	0.922
UKB	3000	Logistic Regression	1:5	0.515	0.071	0.037	0.993
UKB	3000	Logistic Regression	1:10	0.503	0.015	0.007	0.999
UKB	3000	Logistic Regression	1:50	0.500	0.001	0.001	0.999
UKB	3000	MLP	50:1	0.500	0.667	1	0
UKB	3000	MLP	10:1	0.500	0.667	1	0
UKB	3000	MLP	5:1	0.500	0.667	0.999	0.001
UKB	3000	MLP	2:1	0.572	0.675	0.891	0.253
UKB	3000	MLP	1:1	0.620	0.607	0.588	0.651
UKB	3000	MLP	1:2	0.589	0.402	0.285	0.893
UKB	3000	MLP	1:5	0.510	0.052	0.027	0.993
UKB	3000	MLP	1:10	0.500	0	0	1
UKB	3000	MLP	1:50	0.500	0	0	1
UKB	3000	Decision Tree	50:1	0.507	0.665	0.979	0.035
UKB	3000	Decision Tree	10:1	0.528	0.652	0.884	0.173
UKB	3000	Decision Tree	5:1	0.549	0.647	0.828	0.270
UKB	3000	Decision Tree	2:1	0.571	0.624	0.713	0.429
UKB	3000	Decision Tree	1:1	0.581	0.579	0.576	0.587
UKB	3000	Decision Tree	1:2	0.571	0.502	0.432	0.710
UKB	3000	Decision Tree	1:5	0.566	0.399	0.288	0.844

Continued on next page

Continued from previous page

Dataset	Train Size	Model	Pos:Neg	ACC	F1	Sensitivity	Specificity
UKB	3000	Decision Tree	1:10	0.530	0.262	0.167	0.894
UKB	3000	Decision Tree	1:50	0.506	0.080	0.043	0.969
UKB	3000	XGBoost	50:1	0.500	0.667	1	0
UKB	3000	XGBoost	10:1	0.504	0.668	0.997	0.011
UKB	3000	XGBoost	5:1	0.539	0.680	0.982	0.095
UKB	3000	XGBoost	2:1	0.641	0.711	0.883	0.400
UKB	3000	XGBoost	1:1	0.663	0.665	0.669	0.657
UKB	3000	XGBoost	1:2	0.624	0.492	0.365	0.884
UKB	3000	XGBoost	1:5	0.536	0.159	0.088	0.984
UKB	3000	XGBoost	1:10	0.507	0.031	0.016	0.999
UKB	3000	XGBoost	1:50	0.500	0	0	1
UKB	3000	LightGBM	50:1	0.500	0.667	1	0
UKB	3000	LightGBM	10:1	0.503	0.667	0.998	0.009
UKB	3000	LightGBM	5:1	0.545	0.684	0.985	0.105
UKB	3000	LightGBM	2:1	0.643	0.713	0.886	0.400
UKB	3000	LightGBM	1:1	0.675	0.675	0.677	0.672
UKB	3000	LightGBM	1:2	0.629	0.505	0.379	0.879
UKB	3000	LightGBM	1:5	0.545	0.192	0.108	0.982
UKB	3000	LightGBM	1:10	0.506	0.027	0.014	0.999
UKB	3000	LightGBM	1:50	0.500	0	0	1
UKB	3000	FT-Transformer	50:1	0.500	0.667	1	0
UKB	3000	FT-Transformer	10:1	0.500	0.667	1	0
UKB	3000	FT-Transformer	5:1	0.500	0.667	1	0
UKB	3000	FT-Transformer	2:1	0.500	0.667	1	0
UKB	3000	FT-Transformer	1:1	0.500	0.667	1	0
UKB	3000	FT-Transformer	1:2	0.500	0	0	1
UKB	3000	FT-Transformer	1:5	0.500	0	0	1
UKB	3000	FT-Transformer	1:10	0.500	0	0	1
UKB	3000	FT-Transformer	1:50	0.500	0	0	1
UKB	3000	MHL	50:1	0.511	0.653	0.921	0.102
UKB	3000	MHL	10:1	0.521	0.663	0.943	0.099
UKB	3000	MHL	5:1	0.545	0.628	0.795	0.295
UKB	3000	MHL	2:1	0.534	0.582	0.671	0.398
UKB	3000	MHL	1:1	0.561	0.621	0.723	0.399
UKB	3000	MHL	1:2	0.587	0.613	0.667	0.507
UKB	3000	MHL	1:5	0.498	0.640	0.903	0.092
UKB	3000	MHL	1:10	0.559	0.642	0.790	0.327
UKB	3000	MHL	1:50	0.504	0.664	0.979	0.029
CCID	1000	Logistic Regression	50:1	0.508	0.670	0.999	0.017
CCID	1000	Logistic Regression	10:1	0.574	0.695	0.973	0.175
CCID	1000	Logistic Regression	5:1	0.617	0.714	0.954	0.280
CCID	1000	Logistic Regression	2:1	0.723	0.751	0.833	0.613
CCID	1000	Logistic Regression	1:1	0.757	0.743	0.703	0.811
CCID	1000	Logistic Regression	1:2	0.734	0.678	0.561	0.906
CCID	1000	Logistic Regression	1:5	0.642	0.476	0.327	0.958
CCID	1000	Logistic Regression	1:10	0.605	0.366	0.229	0.980
CCID	1000	Logistic Regression	1:50	0.521	0.096	0.051	0.991
CCID	1000	MLP	50:1	0.500	0.667	1	0
CCID	1000	MLP	10:1	0.500	0.667	1	0
CCID	1000	MLP	5:1	0.654	0.727	0.921	0.387
CCID	1000	MLP	2:1	0.730	0.748	0.803	0.657
CCID	1000	MLP	1:1	0.763	0.751	0.715	0.812
CCID	1000	MLP	1:2	0.725	0.669	0.555	0.896
CCID	1000	MLP	1:5	0.571	0.230	0.157	0.986
CCID	1000	MLP	1:10	0.562	0.225	0.138	0.987

Continued on next page

Continued from previous page

Dataset	Train Size	Model	Pos:Neg	ACC	F1	Sensitivity	Specificity
CCID	1000	MLP	1:50	0.500	0	0	1
CCID	1000	Decision Tree	50:1	0.535	0.674	0.961	0.108
CCID	1000	Decision Tree	10:1	0.605	0.689	0.875	0.335
CCID	1000	Decision Tree	5:1	0.624	0.675	0.781	0.467
CCID	1000	Decision Tree	2:1	0.679	0.691	0.721	0.637
CCID	1000	Decision Tree	1:1	0.688	0.686	0.681	0.695
CCID	1000	Decision Tree	1:2	0.671	0.629	0.559	0.783
CCID	1000	Decision Tree	1:5	0.620	0.493	0.370	0.870
CCID	1000	Decision Tree	1:10	0.620	0.454	0.317	0.924
CCID	1000	Decision Tree	1:50	0.537	0.173	0.098	0.975
CCID	1000	XGBoost	50:1	0.503	0.668	0.999	0.008
CCID	1000	XGBoost	10:1	0.613	0.714	0.966	0.261
CCID	1000	XGBoost	5:1	0.690	0.749	0.923	0.457
CCID	1000	XGBoost	2:1	0.751	0.768	0.824	0.678
CCID	1000	XGBoost	1:1	0.761	0.760	0.759	0.763
CCID	1000	XGBoost	1:2	0.746	0.703	0.603	0.889
CCID	1000	XGBoost	1:5	0.646	0.480	0.327	0.966
CCID	1000	XGBoost	1:10	0.582	0.303	0.182	0.982
CCID	1000	XGBoost	1:50	0.508	0.030	0.015	1
CCID	1000	LightGBM	50:1	0.505	0.669	1	0.009
CCID	1000	LightGBM	10:1	0.622	0.721	0.975	0.269
CCID	1000	LightGBM	5:1	0.689	0.747	0.920	0.457
CCID	1000	LightGBM	2:1	0.741	0.757	0.809	0.674
CCID	1000	LightGBM	1:1	0.761	0.760	0.758	0.763
CCID	1000	LightGBM	1:2	0.745	0.701	0.597	0.894
CCID	1000	LightGBM	1:5	0.642	0.469	0.316	0.969
CCID	1000	LightGBM	1:10	0.578	0.295	0.176	0.981
CCID	1000	LightGBM	1:50	0.503	0.011	0.005	1
CCID	1000	FT-Transformer	50:1	0.500	0.667	1	0
CCID	1000	FT-Transformer	10:1	0.500	0.667	1	0
CCID	1000	FT-Transformer	5:1	0.500	0.667	1	0
CCID	1000	FT-Transformer	2:1	0.555	0.689	0.970	0.141
CCID	1000	FT-Transformer	1:1	0.625	0.699	0.857	0.393
CCID	1000	FT-Transformer	1:2	0.672	0.631	0.562	0.782
CCID	1000	FT-Transformer	1:5	0.538	0.140	0.093	0.984
CCID	1000	FT-Transformer	1:10	0.500	0	0	1
CCID	1000	FT-Transformer	1:50	0.500	0	0	1
CCID	1000	MHL	50:1	0.609	0.667	0.787	0.432
CCID	1000	MHL	10:1	0.618	0.670	0.779	0.456
CCID	1000	MHL	5:1	0.610	0.686	0.848	0.373
CCID	1000	MHL	2:1	0.671	0.706	0.792	0.551
CCID	1000	MHL	1:1	0.708	0.728	0.783	0.632
CCID	1000	MHL	1:2	0.664	0.714	0.833	0.496
CCID	1000	MHL	1:5	0.669	0.723	0.861	0.477
CCID	1000	MHL	1:10	0.690	0.719	0.798	0.582
CCID	1000	MHL	1:50	0.675	0.700	0.760	0.589
CCID	3000	Logistic Regression	50:1	0.505	0.668	0.996	0.013
CCID	3000	Logistic Regression	10:1	0.569	0.694	0.976	0.162
CCID	3000	Logistic Regression	5:1	0.629	0.721	0.957	0.301
CCID	3000	Logistic Regression	2:1	0.735	0.761	0.846	0.624
CCID	3000	Logistic Regression	1:1	0.762	0.746	0.700	0.823
CCID	3000	Logistic Regression	1:2	0.728	0.666	0.543	0.914
CCID	3000	Logistic Regression	1:5	0.669	0.532	0.377	0.960
CCID	3000	Logistic Regression	1:10	0.615	0.392	0.249	0.981
CCID	3000	Logistic Regression	1:50	0.526	0.101	0.053	0.999

Continued on next page

Continued from previous page

Dataset	Train Size	Model	Pos:Neg	ACC	F1	Sensitivity	Specificity
CCID	3000	MLP	50:1	0.500	0.667	1	0
CCID	3000	MLP	10:1	0.677	0.736	0.900	0.453
CCID	3000	MLP	5:1	0.721	0.742	0.805	0.637
CCID	3000	MLP	2:1	0.738	0.724	0.690	0.785
CCID	3000	MLP	1:1	0.725	0.678	0.581	0.869
CCID	3000	MLP	1:2	0.705	0.624	0.493	0.917
CCID	3000	MLP	1:5	0.700	0.602	0.457	0.943
CCID	3000	MLP	1:10	0.600	0.329	0.229	0.971
CCID	3000	MLP	1:50	0.504	0.016	0.008	0.999
CCID	3000	Decision Tree	50:1	0.566	0.677	0.909	0.223
CCID	3000	Decision Tree	10:1	0.649	0.683	0.759	0.539
CCID	3000	Decision Tree	5:1	0.651	0.657	0.667	0.635
CCID	3000	Decision Tree	2:1	0.657	0.627	0.577	0.736
CCID	3000	Decision Tree	1:1	0.661	0.602	0.514	0.808
CCID	3000	Decision Tree	1:2	0.659	0.576	0.463	0.855
CCID	3000	Decision Tree	1:5	0.657	0.597	0.507	0.807
CCID	3000	Decision Tree	1:10	0.645	0.525	0.393	0.898
CCID	3000	Decision Tree	1:50	0.565	0.273	0.164	0.965
CCID	3000	XGBoost	50:1	0.541	0.685	0.997	0.085
CCID	3000	XGBoost	10:1	0.715	0.766	0.933	0.497
CCID	3000	XGBoost	5:1	0.744	0.768	0.849	0.638
CCID	3000	XGBoost	2:1	0.770	0.760	0.726	0.815
CCID	3000	XGBoost	1:1	0.754	0.715	0.618	0.891
CCID	3000	XGBoost	1:2	0.718	0.635	0.492	0.944
CCID	3000	XGBoost	1:5	0.718	0.637	0.494	0.942
CCID	3000	XGBoost	1:10	0.632	0.440	0.290	0.974
CCID	3000	XGBoost	1:50	0.516	0.068	0.035	0.997
CCID	3000	LightGBM	50:1	0.544	0.686	0.996	0.092
CCID	3000	LightGBM	10:1	0.717	0.767	0.932	0.502
CCID	3000	LightGBM	5:1	0.755	0.775	0.842	0.668
CCID	3000	LightGBM	2:1	0.766	0.750	0.703	0.829
CCID	3000	LightGBM	1:1	0.750	0.705	0.599	0.901
CCID	3000	LightGBM	1:2	0.717	0.630	0.483	0.951
CCID	3000	LightGBM	1:5	0.717	0.636	0.495	0.939
CCID	3000	LightGBM	1:10	0.637	0.451	0.299	0.975
CCID	3000	LightGBM	1:50	0.513	0.052	0.027	1
CCID	3000	FT-Transformer	50:1	0.500	0.667	1	0
CCID	3000	FT-Transformer	10:1	0.557	0.692	0.981	0.133
CCID	3000	FT-Transformer	5:1	0.693	0.742	0.883	0.504
CCID	3000	FT-Transformer	2:1	0.713	0.751	0.865	0.561
CCID	3000	FT-Transformer	1:1	0.726	0.743	0.788	0.665
CCID	3000	FT-Transformer	1:2	0.670	0.663	0.657	0.684
CCID	3000	FT-Transformer	1:5	0.637	0.485	0.348	0.926
CCID	3000	FT-Transformer	1:10	0.578	0.294	0.197	0.959
CCID	3000	FT-Transformer	1:50	0.500	0	0	1
CCID	3000	MHL	50:1	0.671	0.705	0.788	0.553
CCID	3000	MHL	10:1	0.634	0.695	0.834	0.435
CCID	3000	MHL	5:1	0.646	0.671	0.727	0.565
CCID	3000	MHL	2:1	0.653	0.697	0.797	0.509
CCID	3000	MHL	1:1	0.705	0.727	0.785	0.625
CCID	3000	MHL	1:2	0.639	0.703	0.853	0.426
CCID	3000	MHL	1:5	0.684	0.709	0.765	0.602
CCID	3000	MHL	1:10	0.610	0.688	0.860	0.360
CCID	3000	MHL	1:50	0.710	0.677	0.609	0.811

Table 6: Summary across different LLM backends.

Backend	Dataset	ACC	F1	Sensitivity	Specificity
DeepSeek V4-Flash	UKB	0.556	0.662	0.872	0.239
DeepSeek V4-Pro	UKB	0.583	0.603	0.635	0.531
DeepSeek V4-Pro-Thinking	UKB	0.556	0.601	0.669	0.443
GPT-5.5	UKB	0.545	0.664	0.899	0.192
Gemini 3.1-Pro	UKB	0.606	0.645	0.715	0.497
Qwen 3.7-Max	UKB	0.590	0.639	0.730	0.451
DeepSeek V4-Flash	CCID	0.679	0.699	0.744	0.615
DeepSeek V4-Pro	CCID	0.708	0.728	0.783	0.632
DeepSeek V4-Pro-Thinking	CCID	0.658	0.698	0.789	0.526
GPT-5.5	CCID	0.665	0.721	0.867	0.463
Gemini 3.1-Pro	CCID	0.707	0.731	0.795	0.619
Qwen 3.7-Max	CCID	0.703	0.728	0.797	0.608

Table 7: Continual learning summary on MIMIC.

Model	Stage1 ACC	Stage1 F1	Stage2 ACC	Stage2 F1	Delta F1
Logistic Regression	0.664	0.661	0.563	0.548	-0.113
MLP	0.584	0.620	0.544	0.290	-0.330
Decision Tree	0.580	0.579	0.521	0.500	-0.079
XGBoost	0.664	0.665	0.631	0.626	-0.039
LightGBM	0.661	0.662	0.642	0.648	-0.014
Heuristic Learning	0.579	0.668	0.599	0.683	0.015

B Prompt Templates

B.1 Knowledge Probe Prompt

Source function: `get_knowledge_probe_prompt(...)`

You are a medical prior-knowledge assistant. Generate clinical prior knowledge for the given
 ↳ tabular medical features.
 Write EVERYTHING in English only.
 [Optional if {task_description} is non-empty]
 Task description: {task_description}
 Outcome/target column: {target}
 You MUST return a Markdown table that includes the relationships between the features and the
 ↳ outcome, with exactly these columns:
 | Feature | Univariate signal (summary) | Clinical rationale | Suggested threshold | Evidence
 ↳ confidence (high/medium/low) |
 Requirements:
 - You must provide a suggested threshold (if not applicable, write "no clear threshold" and explain
 ↳ why)
 - Clinical rationale should be specific enough to justify rules
 - Evidence confidence must be one of: high / medium / low
 Feature list (json):
 {features_json}

B.2 Initial Rule Generation Prompt

Source function: `get_rule_generation_prompt(...)`

You are a medical rule learning agent. Based on the input information, you will generate a
 ↳ pure-Python classification rule function.
 Write EVERYTHING in English only.
 Inputs include: univariate summary, medical knowledge table, and metric priority.
 [Optional if {task_description} is non-empty]
 Task description: {task_description}
 {metric_desc}

[Univariate Summary]
 {univariate_summary}

[Medical Knowledge Table]
 {knowledge_table}

Return STRICT JSON with fields:
 - version: "v0"
 - error_analysis: the design rationale (in English)
 - new_policy_code: full Python function definition; function name MUST be predict_v0
 Function signature MUST be: `def predict_v0(features: dict) -> int:`
 Return an integer class label. For binary tasks, return 0/1 matching the dataset label.
 You may implement a score-based rule, a direct rule-based decision tree, or any deterministic rule
 ↳ set, as long as it is a pure-Python function.
 Each if/elif/else branch MUST include an English comment briefly explaining the medical rationale
 ↳ or design intent.
 The rule must be self-contained and use ONLY the Python standard library (no third-party packages).

B.3 Iteration Prompt

Source function: `get_iteration_prompt(...)`

You are a medical rule learning agent. Update the current Python-based classification rule based
 ↳ on:
 - Current full code (all historical versions)
 - This round's training set error analysis
 - Iteration trajectory (reasons for previous changes)
 - Metric priority

- (If any) degradation warning: cases that regressed from correct to wrong after the last change

[Optional if {task_description} is non-empty]
Task description: {task_description}

{metric_desc}

[Current Full Code]
{current_code}

[Training Error Analysis]
{error_report}

[Iteration Trajectory]
{trajectory}

[Degradation Warning]
{degradation_warning}

Important: if a degradation warning exists, you MUST prioritize fixing regressed cases (previously
→ correct, now wrong) and try not to introduce new regressions.

Each if/elif/else branch MUST include an English comment briefly explaining the medical rationale
→ or design intent.

Extra constraint: do NOT collapse to predicting almost all 1s or 0s.

Minimal-change constraint: only make small adjustments this round (e.g., adjust 1-2

→ thresholds/weights, or add/remove no more than 2 rules). Do NOT rewrite the whole function.

Return STRICT JSON:

```
{
  "version": "{next_version}",
  "error_analysis": "...(English)...",
  "new_policy_code": "def predict_...\n ..."
}
```

Keep changes minimal and comments clear.

The rule must be self-contained and use ONLY the Python standard library (no third-party packages).

B.4 Continual Learning v0 Generation Prompt

Source function: get_continuous_v0_generation_prompt(...)

You are a medical rule learning agent. I have already built a pure-Python classification rule
→ function. However, feature drift has occurred. Based on the following summary of feature drift,
→ together with the provided univariate summary, medical knowledge table, metric priority, and
→ other inputs, update the classification rule function.

Write EVERYTHING in English only.

Inputs include: feature drift summary, updated univariate summary, updated medical knowledge table,
→ metric priority, and the previous final model blueprint.

[Optional if {task_description} is non-empty]

Task description: {task_description}

{metric_desc}

[Feature Drift Summary]
- Dropped columns: {dropped_cols}
- Added columns: {added_cols}
- Renamed columns: {renamed_cols}
- Change note: {change_note}

[Optional if {univariate_summary} is non-empty]

[Updated Univariate Summary]

{univariate_summary}

[Optional if {knowledge_table} is non-empty]

[Updated Medical Knowledge Table]

{knowledge_table}

[Previous Final Model Blueprint]

{blueprint_code}

Return STRICT JSON with fields:

- version: "v0"
- error_analysis: the design rationale (in English)
- new_policy_code: full Python function definition; function name MUST be predict_v0

Function signature MUST be: def predict_v0(features: dict) -> int:

Return an integer class label. For binary tasks, return 0/1 matching the dataset label.

Do not reference dropped features.

You may use added or renamed features if useful.

Each if/elif/else branch MUST include an English comment briefly explaining the medical rationale

↔ or design intent.

The rule must be self-contained and use ONLY the Python standard library (no third-party packages).